

FlashANNs: Hiding NAND from Graph Search on Flash-Backed CXL Memory

Anonymous Author(s)

Abstract

Billion-scale graph ANNS needs a flash-priced corpus and a hot path a query host can score from. Block-SSD systems such as DiskANN hide NAND behind asynchronous I/O and PQ navigation, but remain a per-host block serving unit. Full-in-memory CXL-DRAM systems such as CXL-ANNS and Cosmos remove the flash miss entirely and price the cold tail at DRAM. Flash-backed CXL memory is the third point: full-precision vectors remain on Flash, a 4 GiB CXL-DRAM page cache exposes memory-semantic access, and host DRAM keeps the adjacency graph, compact PQ codes, and a smaller scoring window. A unified VA does not make search usable. The runtime question is what must be hidden so that graph walk does not wait on NAND.

We present FlashANNs, a graph-ANNS runtime that splits that contract. *Score hide* permits a host-cache or CXL-DRAM-cache hit, but no scoring access may trigger or wait for a Flash fill. *Query hide*—the timed path also does not wait for a fill—is the oracle, not a result we claim. Host-side PQ navigation first commits a bounded rerank set; wise prefetching then fetches only its missing full-precision vector pages, using co-use-aware placement and dense page requests to increase useful bytes per read, while continuous batching issues such fills across queries so one walk does not collapse device depth to one. The evaluation separates host-cache hits, CXL-DRAM hits, and Flash fills at iso-recall, and compares the three-tier path with demand access, block-SSD search, and a separate full-corpus CXL-DRAM Oracle. A same-machine PipeANN block path provides a named block-I/O comparison at matched recall. We evaluate a dual-NVMe Flash backing through the software memory-semantic manager `vmem_sw`; we do not equate that manager with a product CXL.mem SSD or claim 160 GB/s, and we treat multi-host one-copy as an unevaluated Shared-FAM deployment model.

CCS Concepts: • Computer systems organization → Secondary storage organization; • Information systems → Nearest-neighbor search; • Software and its engineering → Memory management.

Keywords: CXL, CXL-SSD, approximate nearest neighbor search, graph ANNS, memory-semantic flash, residency, prefetch, continuous batching

1 Introduction

Graph-based approximate nearest neighbor search (ANNS) is a production serving primitive: a query walks a proximity graph and scores neighbors until a beam budget is spent. At billions of vectors the full-precision embeddings no longer fit in host DRAM, even when the more compact graph and PQ metadata do, so the vector payload must live behind another tier. CXL makes that tier addressable with `load/store`. When the whole corpus fits in CXL-DRAM, prior systems already offer a usable service—CXL-ANNS and Cosmos [15, 20] never stall on flash. This paper is about the next device. Flash-backed CXL memory exposes Flash capacity through a bounded CXL-DRAM page cache on a CXL.mem-style address space [6, 22, 41, 42]. A query host may keep a still smaller software cache, but a miss in the CXL-DRAM cache costs tens of microseconds because it fills from Flash. To our knowledge, FlashANNs is the first *graph-ANNS serving runtime* that operates directly over an SSD-backed CXL address space and enforces *score hide*. The host computes from its small cache or from a CXL-DRAM-cache hit; no scoring access may trigger or wait for a Flash fill. We do not claim the first CXL-DRAM ANNS, the first CXL-SSD attached to a vector database, or a product CXL.mem SSD.

Two published lines each solve one side of the capacity-latency tension and tax the other (Figure 1). DiskANN [31, 32] and pipelined descendants such as PipeANN [13] are excellent *block-SSD* ANNS: PQ navigation, sector layout, and asynchronous I/O hide NAND behind a per-host cache. Their serving unit remains CPU + DRAM working set + a local SSD index. Scaling the service across hosts commonly replicates or shards the flash-resident index. They hide *block I/O*, not a memory-semantic hierarchy with two managed cache levels. CXL-ANNS and Cosmos put the graph and embeddings in a CXL-DRAM pool. Search never sees flash, but the cold tail $D - H$ is priced at DRAM [5, 19]. We build a third service because neither line is the right contract for flash-backed CXL memory: full-precision vectors should stay Flash-priced, compact search information should guide movement from host DRAM, and the runtime must hide Flash fills from the scoring instruction. Multi-host one-copy under Shared FAM is a deployment path we do not evaluate (§2).

Making that service real is hard because the device is not slightly slow DRAM and not a DiskANN SSD.

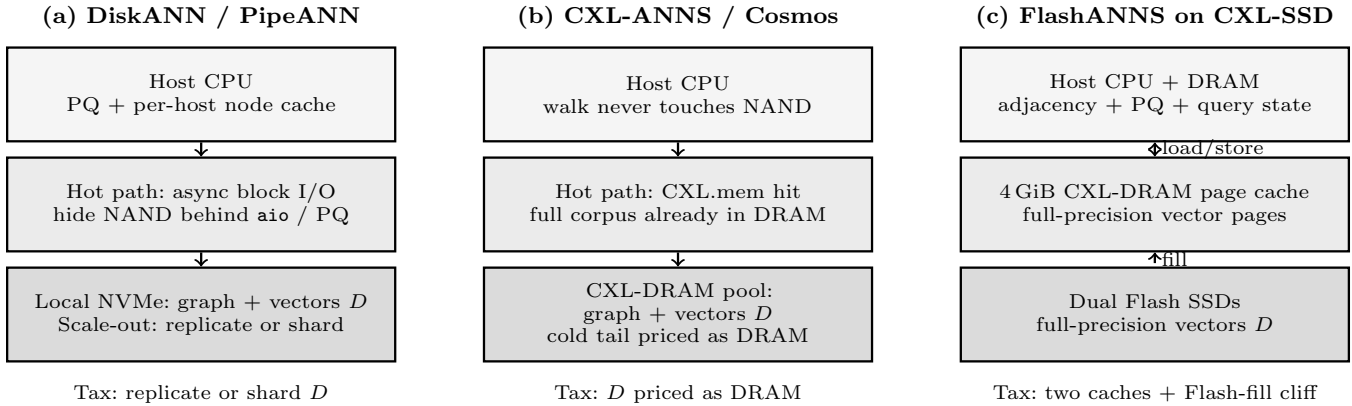


Figure 1. Two published deployments and the third point this paper serves. Flash-backed CXL memory is the *opportunity*: Flash-priced D , a 4 GiB CXL-DRAM page cache, and a small host cache. FlashANNS is the *runtime*: a wise prefetcher plus continuous batching so scoring does not wait on NAND. Host DRAM holds search information; the two lower tiers cache and store the full-precision payload. Shared FAM is not a measured link.

Three hardware-search interactions break conventional designs (§3). (C1) *The miss cliff*. A CXL-DRAM-cache hit is a DRAM-class access; on our measured path, a Flash fill takes about $89\mu\text{s}$, or $130\text{--}320\times$ the latency of a CXL-DRAM item load. Graph walk multiplies the tax by hops $\times$ degree, so treating the composed hierarchy as uniform mmap’d memory makes search miss-bound. (C2) *Prefetch must be wise*. Neighbor IDs are not sequential. OS-style page admission and synchronous 2-hop prefetch bring records the walk will not use, pollute the 4 GiB cache, and can drop QPS by $3\text{--}5\times$. Wrong pages cost more than a missed page. (C3) *One query cannot fill the device*. Best-first search cannot issue hop $i+1$ ’s vector pages until hop i scores. A blocking CXL-DRAM-cache miss collapses queue depth to ≈ 1 ; a single walk leaves the Flash pipeline idle even when bandwidth remains. Continuous issue across queries is what holds depth; it does not, by itself, make one walk as fast as an in-window oracle.

FlashANNS is a runtime for those three constraints, not a new graph index. A placement premise comes first: host DRAM holds graph adjacency and compact PQ codes, while full-precision vectors remain in the Flash-backed address space behind a bounded CXL-DRAM page cache. The first mechanism is a *wise prefetcher*. After PQ navigation commits the rerank candidates, it maps only their missing full-precision vectors to unique pages and issues those pages before exact scoring. An offline co-use-aware layout, online page deduplication, and density-bounded coalescing increase the fraction of each transferred page that reranking actually consumes; the prefetcher neither walks a speculative 2-hop neighborhood nor scores unused siblings to inflate utilization. The second mechanism is *continuous batching*. Fills are

issued at fetch granularity across in-flight queries so that when one walk is stuck on pointer-chasing, another query keeps the device busy. Prefetch and batching may trigger Flash fills; the scoring thread may access the host cache or CXL-DRAM, but may not trigger or wait for Flash. We call that split *score hide*. The stronger contract—*query hide*, no wait for a fill on the timed path—is the oracle, not a result we claim. Our evaluation therefore reports host-cache hits, CXL-DRAM hits, score-triggered Flash fills, and critical-path Flash wait before reporting iso-recall throughput. The separate full-corpus CXL-DRAM Oracle removes Flash from the timed path; it is not a second active copy in the normal 4 GiB-cache configuration.

This paper makes four contributions:

1. **A third serving point, scoped.** We place graph ANNS on a three-tier host-cache/CXL-DRAM-cache/Flash hierarchy and separate the *opportunity* (Flash-priced D) from the *system* (hide Flash fills from scoring). We do not claim the first CXL-DRAM ANNS or an evaluated Shared-FAM deployment (§2).
2. **Why naive CXL-SSD ANNS fails.** We measure the miss cliff, unwise page/prefetch admission, and single-query queue collapse, and we turn those negatives into design constraints (§3).
3. **A wise prefetcher plus continuous batching.** The prefetcher waits for candidate commitment, fetches only missing rerank-vector pages, and shapes them for high useful-page occupancy; batching holds device depth across walks. Score hide requires zero score-triggered Flash fills, while the full-corpus CXL-DRAM Oracle remains a separate upper bound (§4, §5).

4. **An iso-recall evaluation of hide, not just QPS.** We report the three-level hit/fill breakdown and critical-path Flash wait before iso-recall latency and throughput. Same-machine PipeANN is a named block-I/O comparison, not a hidden bar (§6).

2 Background

2.1 Graph ANNS walk

A graph ANNS index (Vamana [32], HNSW [25], or NSG [7]) stores, for each vector, a neighbor list of degree R . Search keeps a candidate set of width L (beam / **efSearch**). A *hop* expands the closest unread candidate u : the walk reads $N(u)$, scores those neighbors, and inserts the best unseen IDs back into the candidate set. The query returns the top- k after a hop or candidate budget is spent. Product quantization and its optimized variants compress the vectors used for approximate candidate ranking [8, 16]; FAISS and ScaNN provide widely used implementations of this quantized-search design space [14, 17].

One expand touches two kinds of bytes. Neighbor *IDs* are small and can live in host DRAM. Neighbor *vectors* are large: on Text2Image-10M each is 800 B, so scoring $N(u)$ moves about $R \times 800$ B if every neighbor is new. A typical hop is:

1. pick the closest unread candidate u from the beam;
2. read $N(u)$ (IDs only);
3. score the unseen neighbors' full-precision vectors;
4. insert the best new IDs into the candidate set.

Two facts follow for a memory hierarchy. First, the walk is *pointer-chasing*: step 3 of hop i decides which vector pages hop $i+1$ will need. Second, IDs need not live with vectors—the adjacency list can sit in host DRAM while embeddings sit on a slower tier. Recall depends on which useful neighbors the budget scores; stall rises with how many of those scores miss the fast tier. Widening L scores more nodes and cools the working set; that coupling is a measurement in §3, not a claim here.

2.2 The block-SSD serving stack

DiskANN and its maintained family [21, 31, 32] are the reference block-SSD design. Full-precision vectors and adjacency share 4 KB-aligned NVMe sectors. DRAM-resident PQ codes rank candidates without fetching full-precision vectors; an **aio** beam issues several sector reads at once; a static node cache retains entry and frequently visited vertices. PQ navigation exists to *avoid* full-precision reads, not to make NAND faster. Asynchronous I/O *hides* remaining reads by overlapping them with other work. Because graph and vectors share a sector, one 4 KB read is bound to whatever else the layout packed into that page.

Table 1. Latency tiers that a graph walk can see. CXL-DRAM is the published range; NAND fill is the order on the path we measure. The cliff itself is §3.

Access	Scale	Where
Host DRAM	~ 100 ns	graph / PQ metadata
Score-window hit	DRAM class	resident vectors
CXL-DRAM	140–410 ns	pooled graph / vectors
NAND fill	tens of μ s	miss $\rightarrow$ flash

PipeANN [13] keeps that layout, overlaps compute with I/O inside one query, and can interleave queries to hold device queue depth. That cross-query interleaving is the block analogue of the continuous issue used later: its objects are I/O requests and sectors. The same idea will reappear on CXL-SSD as fetch-level issue (§4), against a faulting address space rather than **aio**.

2.3 CXL-DRAM ANNS

CXL.mem maps device HDM into the host physical address space [6]. Published Type-3 CXL-DRAM latencies sit in the 140–410 ns range [15, 33]. CXL-ANNS places the graph and embeddings in a CXL-DRAM pool and hides that far-memory delay with a relationship-aware cache and DSA-assisted prefetch. Cosmos [20] further offloads traversal and distance to device-side compute. Second-tier and residual designs [5, 28, 43] likewise assume a memory-class far tier without a per-miss NAND fill.

The interface is **load/store**; the capacity assumption is that D fits in CXL-DRAM. A service on that line is already usable in the sense that search never waits on flash. Prefetch and cache policies tuned for hundreds of nanoseconds do not transfer to a NAND fill of tens of microseconds. We keep the detailed contrast in §7.

2.4 Flash-backed CXL memory

In the device model studied here, a CXL-SSD pairs NAND capacity with a byte-addressable window [18, 22, 39, 41, 42]. Figure 1c is the deployment we serve. A *score-window hit* is served at DRAM-class latency. A *miss* is a NAND fill into that window, then a retry. The programming model looks like memory; the miss tax looks like flash. Table 1 places the three published tiers next to the NAND fill this paper must hide. The measured cliff belongs in §3; the table only fixes the orders of magnitude.

The window is bounded, and the admission unit is a page. On Text2Image-10M a vector is 800 B, so a 4 KB page holds several records—or several unused siblings. A blocking **load/store** miss and an explicit prefetch ioctl are both fills; they differ in *who waits*. If the scoring thread issues the fill, NAND is on the critical path.

An early prefetch can overlap the fill; only a fill that completes before scoring leaves the scoring path. That split is the runtime contract in §4, not a result.

The prototype we measure is a software memory-semantic node (`vmem_sw`) in front of NVMe, with the score window implemented as host DRAM (`mbind`). It preserves fill semantics (a miss moves NAND bytes into the window), but not product CXL.mem timing. Hardware and generic OS work on this tier does not manage graph-ANNS residency; those systems belong in §7. Implementation limits belong in §5. Direct-attach CXL-SSD already gives one host flash-priced memory semantics; that is the deployment we measure. Multi-host one-copy under CXL 3.x Shared FAM is a premise we do not evaluate.

3 Motivation

Section 2 defined the walk and the fill. This section measures what happens if that fill is treated as a slightly slow DRAM load or as a DiskANN sector read. Each measurement asks two questions: does the effect exist, and what throughput or NAND-traffic penalty does it cause? The three results are the constraints Intro named C1–C3 (Table 2).

3.1 The miss cliff

A host load of one vector from CXL-DRAM is a few hundred nanoseconds. A matched-size CXL-SSD window-miss read is a NAND fill (Figure 2). On Text2Image-10M (800 B) the p_{50} is $0.28\mu\text{s}$ versus $89\mu\text{s}$ ($\sim 320\times$). On a 10M-item LAION slice (2048 B) it is $0.67\mu\text{s}$ versus $89\mu\text{s}$ ($\sim 130\times$). The miss is a 4 KB fill, so record size barely moves it; the CXL-DRAM hit scales with bytes.

A graph walk can pay that tax across hops and newly scored neighbors. Once a modest fraction of scores miss, wall-clock time is in the NAND path, not in distance arithmetic. DiskANN already knew the cliff for block I/O. The relevant distinction is that a memory-semantic interface does not remove it: `load/store` only changes *how* a miss is issued (§3.3). Treating CXL-SSD as memory-mapped DRAM therefore makes search miss-bound. The runtime contract that follows is not “make NAND faster”; it is that the scoring instruction must not wait on a fill.

3.2 Prefetch must be wise

Neighbor IDs are not sequential in the address space, so the next vector page is not the next byte. Coalescing four 4 KB pages into one 16 KB admission unit therefore brings siblings the walk will not score. On a high-dimensional corpus where a record already fills most of a page, per-record admission versus page admission splits QPS by $\sim 3.6\times$ and NAND reads by $\sim 3.7\times$ (Figure 3a). At small dimensions several records share a page and

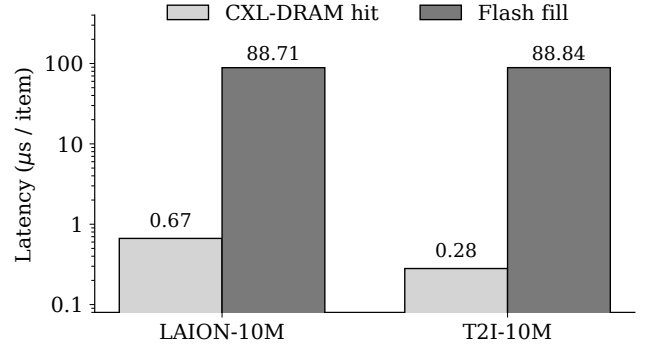


Figure 2. One-item host load: CXL-DRAM hit versus CXL-SSD cache miss. CXL-DRAM is Montage /dev/dax0.0 (`clflush` then load). CXL-SSD is a `vmem_sw` window miss. T2I-10M uses the staged 800 B vectors. LAION-10M is the first 10M items of the 25M dump ($d=512$); the miss bar matches that item size on the same SSD path.

the gap narrows; page-cache thinking is still the wrong default here.

Synchronous two-hop prefetch rooted at the top-1, top-2, or top-8 frontier candidates does not fix the cliff. Hit rate is almost unchanged, NAND reads multiply, and QPS drops by $\sim 4\text{--}5\times$ at the widest setting (Figure 3b). Precision (*used/promoted*) falls as coverage grows; left-over pages evict useful ones from a small window. Wrong pages cost more than a missed page. A page can be necessary yet still waste most of its vector slots. The useful policy therefore delays full-precision movement until the approximate search commits its rerank set, then fetches only that payload and packs co-used vectors together—not “prefetch harder.”

Widening L can become another form of unwise coverage. Under the fixed hop budget, recall and NAND reads both plateau from $L=32$ to 64. Extra width buys no measured quality benefit in that interval; quality and I/O cost must therefore be reported together.

3.3 One query cannot fill the device

Best-first search is pointer-chasing: hop i must score before the walk knows which vector pages hop $i+1$ needs. A single query therefore leaves device-queue bubbles—the beam fill stream ramps slowly, the next fill is unknown, and the walk drains at the end (Figure 4). A block interface exposes I/O requests that a scheduler can interleave across independent queries. On CXL-SSD the programming model is a faulting address space. A blocking `load/store` (or a blocking `memcpy` from the mapping) is a synchronous NAND fault: effective queue depth collapses to ≈ 1 per thread. If a query waits for

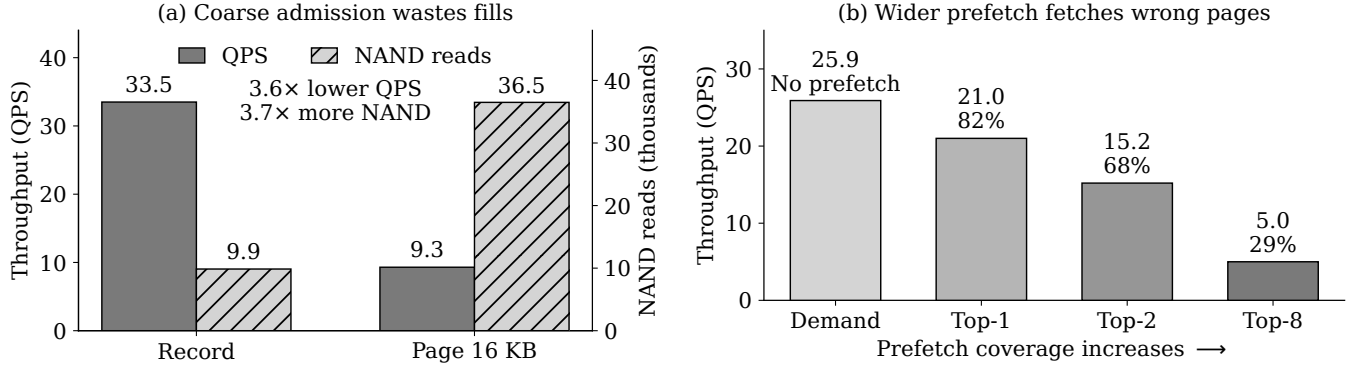


Figure 3. Unwise admission amplifies the cliff (LAION-200k, $d=1024$, hops=32). (a) A 16-KB admission unit generates 3.7× more NAND reads and cuts throughput by 3.6× relative to record admission. (b) Widening synchronous prefetch lowers promotion precision from 82% to 29% and throughput from 21.0 to 5.0 QPS; the unchanged 58% hit rate is omitted.

those fills before scoring, NAND latency remains on its critical path.

The idle device is a pipeline problem, not a bandwidth wall. Continuous issue across queries is what can hold depth; it does not, by itself, make one walk as fast as an in-window oracle, and it does not change which nodes the walk visits. The implementation of that issue belongs in §4.

3.4 From measurements to constraints

Table 2 is the contract for §4. We do not treat CXL-SSD as uniform memory (C1): scoring must not touch NAND. Prefetch must be wise (C2): trigger at candidate commitment, fetch only the committed full-precision payload, and place co-used vectors so each page carries more data that reranking consumes. One walk cannot fill the device (C3): issue must be asynchronous and cross-query so pointer-chasing bubbles do not collapse queue depth.¹ Placement of neighbor IDs in host DRAM is a premise for C2, not a separate challenge: without it the prefetcher misses once just to learn whom to fetch.

4 Design

FlashANNS is a userspace ANNS runtime plus a thin residency plane in front of host DRAM, a bounded window that plays the role of CXL-DRAM, and a flash-backed CXL address space that holds D (Figure 5). Search is still best-first graph walk. The service contract is a *split*: prefetch and continuous batching may touch CXL-SSD; the scoring thread may not. A hop that computes a full-precision distance from bytes that were not already in the window is a hide failure, not a successful promote.

¹A small shared pin of entry vertices helps a narrow beam more than deepening one walk. That is an admission hint, not a positive LFU-hot-set module, and not a fourth challenge.

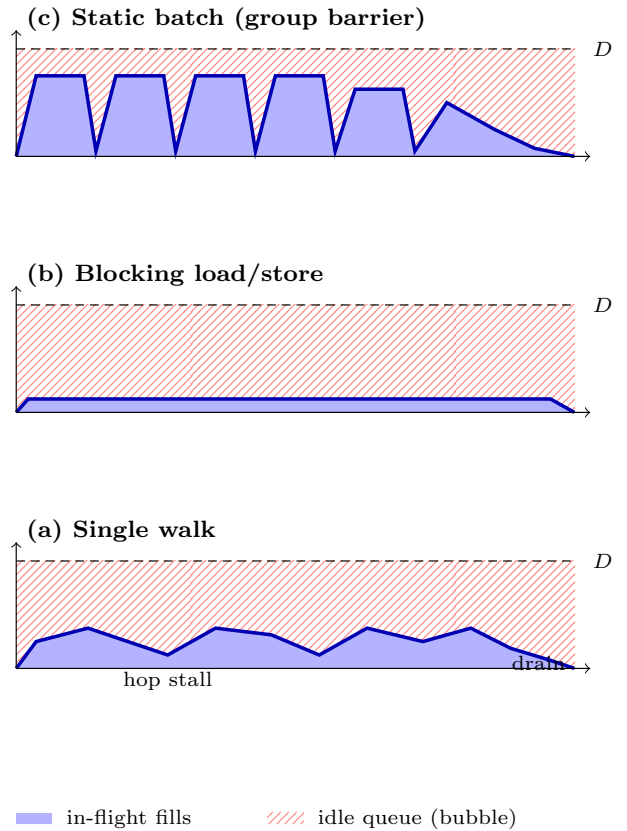


Figure 4. Pipeline bubbles on a faulting CXL address space (schematic). y is in-flight fills; D is the depth NAND parallelism needs. A single walk cannot hold D ; a blocking load/store is stuck at $QD \approx 1$; a static batch drains at the barrier. How the runtime issues across queries is §4.

Table 2. Measured pathologies become the three constraints Intro named. C1 uses the 10M-item probes in Figure 2; C2 uses LAION-200k; C3 follows from the dependency and issue path illustrated above.

ID	Exists + hurts	Constraint	FlashANNS answers
C1	Miss cliff $\sim 130 \times$	Score only resident bytes	Score-hide contract
C2	Page $\sim 3.6 \times$; blind prefetch $\sim 4\text{--}5 \times$	Commit, select, and pack useful pages	Wise prefetcher
C3	Pointer-chasing; blocking fill $\Rightarrow$ QD ≈ 1	Hold depth across walks	Continuous batching

One walk cannot fully hide hop $i+1$ until hop i scores (pointer-chasing). The wise prefetcher instead turns a committed PQ rerank set into a precise page batch before full-precision scoring. Inter-query continuous issue is what keeps device depth up while a single walk is stuck. We do not claim the measured prototype has met this contract (§6); the modules below are the mechanisms that must.

4.1 D1: Semantic placement

Table 3 is the default map. The graph adjacency and the PQ codebook are touched on every hop and are small relative to the full-precision corpus; they live in host DRAM so they never compete with vectors for the CXL-SSD window (F2, F7). Query state (candidates, visited set, thread-local arenas) is likewise host-local. Full-precision vectors default to the CXL-SSD address space. A bounded *hub* / *shared hot set* of vector pages is explicitly cached into the window or into CXL-DRAM (§4.3). Only full-precision vectors in a committed rerank set become prefetch candidates; a discovered frontier does not (F3).

This is a deliberate difference from DiskANN, where adjacency and vectors share SSD sectors. We pay a modest host-DRAM tax for the graph in order to stop the window from turning into an accidental adjacency cache.

4.2 D2: Wise prefetch at candidate commitment

When: after commitment. The host first completes its approximate beam using the adjacency graph and PQ codes, without reading full-precision vectors from Flash. When that beam commits the bounded rerank set C_L , the prefetcher issues immediately: after uncertainty has been pruned, but before exact scoring dereferences

Table 3. Default placement. **Fill-in:** bytes on the corpus you report (e.g., $25\text{M} \times \text{dim}$).

Object	Default tier	Why
Graph (R neighbors)	host DRAM	every hop; keep off the window
PQ / codebook	host DRAM	tiny, extremely hot
Query / visited	host DRAM	per-worker state
Full-precision vec.	CXL-SSD	capacity
Entry / soft-pin	window (tiny)	F7; LFU hot set demoted
Committed rerank vec.	window via prefetch	F3, F5; score only if resident

any vector in C_L . This trigger deliberately gives up speculative lead time to avoid moving payload for candidates that will not survive. The single-query path may still wait for materialization to complete; the invariant is that the subsequent scoring loads do not themselves fault on Flash.

What: the committed payload. The residency plane maps the IDs in C_L to their full-precision vector pages, removes pages that are resident or already in flight, and deduplicates the rest. Every requested page is therefore justified by at least one vector that exact reranking will consume. Adjacency, PQ codes, speculative neighbors, and unused frontier records are not part of this wave. Because Flash moves 4 KiB pages, however, a justified page may still carry vector slots that this query will not score.

How: make each page useful. Offline, a co-use-aware layout groups vectors that disjoint training searches tend to request together and places the resulting pages into short runs. Online, exact page filtering avoids duplicate movement, and the runtime fills gaps only in a short, sufficiently dense run; it never expands a sparse request into a whole SSD stripe. This combination targets useful payload per read rather than batch size alone. Unlike generic remote-memory or device-side sequential prefetchers [23, 26], the request is justified by the committed rerank set rather than an inferred address stream. We distinguish page precision (a fetched page contains any scored vector), vector-slot utilization (scored slots over all slots read), and extent utilization (requested pages over pages transferred after coalescing); vector-slot utilization is the direct measure of real in-page use.

4.3 D3: Entry pin and soft-pin (hot set demoted)

Single-query deepening is a bad way to spend a small window (F4, F7). A 64 MiB shared LFU set raised hit

Fig. 6 placeholder — architecture:
 top: Placement | Window admit | Prefetch+cont | L ctrl;
 middle: residency plane (fill off crit path / score from window);
 bottom: host DRAM (graph/PQ) | window = virtual CXL-DRAM | CXL-SSD.

Figure 5. FlashANNS architecture. From the host, scored bytes look like CXL-DRAM; capacity is CXL-SSD. The residency plane is the only path that moves full-precision bytes. Search never page-faults a neighbor “to be helpful.” **Fill-in:** use shipped names (PREFETCH_BATCH, DramWindow); mark `vmem_sw`.

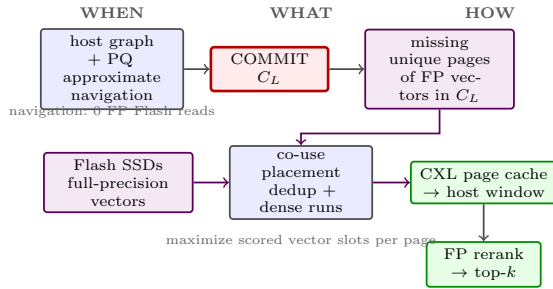


Figure 6. Wise prefetch answers three questions. *When:* after PQ navigation commits C_L . *What:* only missing unique pages of its full-precision vectors. *How:* co-use-aware placement, exact filtering, and dense short runs increase useful vector slots per Flash read.

rate and *destroyed* QPS on a warm protocol (14.3→2.4); we do not ship it as D3. What remains is a tiny hard pin of the entry subgraph (≤ 32 MiB) and a hop-decaying soft-pin of recently installed pages. Pinning is never implicit in a page fault. Cross-query reuse, if it happens, is because committed materialization left useful pages in the window—not because an LFU daemon is competing with search.

4.4 D4: Hide engine — wise prefetch and continuous batching

D4 is the hide engine, not a QPS knob. Offline, we pick the smallest beam L that meets the recall@ k floor. Online, D2 converts each query’s committed rerank set into a precise page batch; the runtime issues that batch before full-precision scoring. Continuous fetch-level batching keeps a target device depth D across queries so one pointer-chasing walk does not collapse to $QD \approx 1$ (Figure 7). Like continuous scheduling in other staged serving systems [40], it admits work at the dependency boundary; here the scheduled objects are committed page fills rather than model iterations. Scoring of already-resident records is never queued behind a fill it does not depend

on. We do not merge the PipeANN binary; its `cont` path is the block-device contrast, not this runtime.

4.5 D5: Observability

Every experiment in §6 is required to report hide counters first: fraction of full-precision scores whose bytes were already in the window (`score_from_window`), critical-path NAND wait (`crit_wait_ns`), and device-fill/wall overlap, then QPS at the iso-recall floor. Prefetch efficiency is reported as page precision, vector-slot utilization, and extent utilization; a page touched once is not evidence that all of its payload was useful. QPS without those counters is not a hide result.

What we refuse to hide. If a module cannot move `crit_wait_ns` or `score_from_window` in an ablation, it is removed from the contribution list rather than kept as decoration. An LFU hot set that raises hit rate and drops QPS is already in that bin.

5 Implementation

FlashANNS is a userspace beam search with the adjacency graph and 64-byte PQ codes in host DRAM and full-precision vectors mapped from `/dev/vmem0`. The host completes PQ navigation, commits the L rerank candidates, maps them through an ID-to-slot table, and removes resident and duplicate 4 KiB pages. The remaining pages form one `PREFETCH_BATCH` wave; dense short runs may include bounded holes, and `PageCopyPool` installs completed pages into `DramWindow` before full-precision MIPS reranking. The explicit residency barrier leaves materialization wait on the query path, but prevents the subsequent scoring loads from triggering Flash faults.

What we measure (2026-08-30, `gpu01`). Kernel 6.18.0-rc5. FPGA Altera 0000:15:00.0 bound to `ntcx/mem2nvme` (32 GiB BAR). NAND I/O goes through `nvme` on 0000:d9:00:00 (→ `/dev/nvme4n1`, Dell CD8P 1.92 TB). Userspace `/dev/vmem0` is `vmem_sw`: 28 GiB logical RAM map, 4 GiB device cache,

Fig. 8 placeholder — miss pipeline:
 queries $\rightarrow$ fetch-level scheduler (target D) $\rightarrow$ async CXL-SSD fills $\rightarrow$ host score $\rightarrow$ admit next.
 Show why blocking load/store cannot hold D .

Figure 7. Continuous fetch-level batching over asynchronous memory-semantic fills. Recall is unchanged by construction: prefetch only changes *when* a needed page is resident, not *which* nodes the walk visits. **Fill-in:** draw the implemented **cont** path; put the blocking-mmap failure mode on the same figure.

2 MiB stripe. HPS registers return `-EOPNOTSUPP`; stores to the BAR read back as `0xff`. Hardware `vmem.ko` implements `PREFETCH_BATCH` but is **not** the measured path. CXL-DRAM DAX is absent; DramWindow is anonymous pages bound to NUMA node 1.

What is implemented. P0 demand access and P3 committed-candidate materialization; host graph and PQ-64 navigation; `PREFETCH_BATCH`; co-use/extent layout with an ID-to-slot map; and the iso-recall L picker. Entry subgraph ≤ 32 MiB is pinned. A shared LFU hot set was prototyped and *lost* QPS on the warm protocol; we do not ship it as a result.

What is not implemented, and is not claimed. Product CXL.mem SSD; HPS fill of the FPGA BAR; Shared FAM; a 160 GB/s DRAM-SSD engine; closed-loop F9 miss-chop; multi-thread shared-window speedup. DiskANN PQ-off is never placed on the PQ-on line.

Prototype scale. The serving binary is a single translation unit plus small headers on top of the CXAN layout. The software CXL-SSD path is the out-of-tree `vmem_sw` driver. A log header that omits kernel, BDF, and `vmem_sw` vs. `vmem.ko` is an invalid row.

6 Evaluation

We answer five questions:

- Q1** Does naive CXL-SSD graph ANNS fail in the ways §3 claimed?
- Q2** At iso-recall, does FlashANNS move FP scoring off the NAND critical path (hide), and what QPS remains vs. demand and vs. an in-window oracle?
- Q3** Which Wise Prefetcher decision (when, what, and how) and how much continuous batching move *hide* (critical-path wait, score-from-window), not only QPS?
- Q4** Does *hide* appear only after warmup (service *hide*), or also on a true-cold reload?
- Q5** What remains vs. an in-window oracle and vs. PipeANN, and is the residual honest?

Table 4. Locked evaluation platform (2026-08-30, gpu01).

Item	Configuration
Host CPU / DRAM	dual-socket Xeon, $\sim 64+64$ GiB host DRAM
CXL-DRAM window	absent ; 1 GiB DramWindow = <code>mbind</code> node 1
CXL-SSD	FPGA <code>nvmeX+CD8P</code> ; <code>/dev/vmem0=vmem_sw</code>
Window / cache	host scoring window; bounded <code>vmem_sw</code> cache (reported per run)
Block-SSD base-line	PipeANN on <code>/mnt/disk0 (nvme0n1)</code> , PQ on
Primary corpus	Text2Image-10M, 200-d, MIPS
Sensitivity	default vs. co-use/extent layout; L picker
Index	Vamana $R=32$, host graph + PQ-64 navigation, committed FP rerank
Target quality	$\text{recall}@10 \geq 0.92$ (picker chose $L=400$)
Protocol	cold: <code>rmmmod vmem_sw</code> until <code>cache_used = 0</code>

6.1 Setup

We follow established ANNS evaluation practice [2, 30]: every performance comparison reports recall and uses matched-recall operating points when claiming a latency or throughput advantage.

Table 4 is the locked configuration. The primary metric is *iso-recall QPS* (and p99) at a stated $\text{recall}@k$ floor. We always report NAND reads, promote bytes, and the three-level hit breakdown. A run that does not declare cold vs. warm is discarded: the window is small enough that accidental warm-up invents speedups.

Baselines. (1) **Demand:** CXL-SSD load/store, no promote. (2) **Page:** 16 KB admission. (3) **Sync-2hop:** the F3 policy. (4) **Static-wide:** large beam, no controller. (5) **Miss-chop:** cut beam on instantaneous miss (the F9 policy). (6) **DiskANN/PipeANN:** official block path, same machine, same corpus, same recall floor; PQ

on unless a figure is labeled `--no.pq.nav.` (7) **Oracle-in-window:** pin the measured working set in the 1 GiB window, then search with no SSD fills (the hide upper bound). This is not DiskANN.

We never present a home-grown `O_DIRECT` reader as “DiskANN.”

6.2 Q1: Naive CXL-SSD fails

Figure 2–4 are reproduced on the evaluation machine under the Table 4 protocol so that Design and Evaluation share one dataset. We expect the same qualitative shape: a two-order cliff, page admission and blind prefetch losing to demand, beam cooling the set, and blocking load/store stuck near $QD \approx 1$. If a finding does not reproduce, it is dropped from the intro, not explained away in a footnote.

6.3 Q2: End-to-end iso-recall (the money plot)

Figure 8 and Table 5 are the submission’s main result. The committed-path cells stay blank until matched repetitions freeze recall, mean and tail latency, QPS, NAND traffic, and residency counters under one code and cache state. We report the residual to both the full-corpus Oracle and same-machine PipeANN; neither a stale oneshot result nor page touch alone is substituted for the new Wise Prefetcher path.

6.4 Q3: Each module is necessary

Figure 9 and Table 6 are the falsifiability test for contribution 3. The first two comparisons answer when and what to prefetch; the layout and coalescing comparisons answer how much of each fetched page is truly consumed; the final row isolates continuous batching without changing the prefetch set.

6.5 Q4: Cold hide versus service hide

Figure 10 is the service question. A production ANNS process is allowed a discarded warmup; a paper row is not allowed to confuse that with a true-cold reload. If `crit.wait` collapses only on the warm pass, we say so in the caption and do not migrate the sentence into §1. The one-copy capacity sketch of §2 remains a model; we do not plot it as a measured Q4.

6.6 Q5: Losses, overhead, and fairness

We report the cases FlashANNS loses or looks worse: warm working sets that already fit the window (async prefetch $\approx$ demand); low dimension where $page \approx record$; the residual gap to PQ-navigated DiskANN; and software-proxy artifacts (global cache lock, QD ceiling). Host-DRAM consumed by the graph and PQ is a cost, not a footnote. CPU is reported because DiskANN on this class of machine is I/O-bound, not CPU-bound; if FlashANNS becomes CPU-bound we say so.

Takeaway. Q2–Q4 must be answerable from hide counters, not from QPS prose. If the money plot needs a verbal caveat that changes the claim, the claim in §1 is the thing to change.

7 Related Work

Table 7 is the comparison we want a reviewer to use. Prose below only records the one-line difference; it does not repeat the background.

Block-SSD ANNS. DiskANN, FreshDiskANN, and PipeANN optimize PQ navigation and I/O overlap on block devices [13, 31, 32]. SPANN and Starling provide alternative hybrid and disk-resident layouts [4, 36]. SPFresh and Filtered-DiskANN extend this family toward updates and filtered search [9, 38]; production vector systems such as Milvus [35] broaden the serving context. FlashANNS is not “a better `io_uring` scheduler.” Its first-class objects are a CXL-DRAM-shaped window and the committed vector-page fills that must finish before exact reranking.

CXL-DRAM and near-memory ANNS. CXL-ANNS and Cosmos [15, 20] — and second-tier / residual designs [5, 28, 43] — assume a DRAM- or NVM-priced far tier and optimize for ~ 100 ns, not for tens of microseconds of NAND plus a small polluted window. Recent CXL-aware vector-database restructuring makes the same memory-class tier assumption [19]. Their caches and prefetchers are not transferable at that delay gap (F3). We do not require Type-2 / PNM compute.

Far-memory runtimes and CXL tiering. Remote paging systems manage capacity through swapping and isolation [12, 34], while application-integrated runtimes expose finer-grained control and prefetch [3, 26, 29]. CXL tiering systems instead migrate or place hot pages across DRAM-class tiers [27, 37, 44, 45] and study the resulting throughput/capacity tradeoff [1, 24]. These policies react to generic access locality; FlashANNS uses PQ candidate commitment to identify the exact full-precision payload required by reranking.

CXL-SSD hardware and generic software. DirectCXL and CXL memory pooling establish direct and pooled memory expansion [10, 11]; CXL-enabled SSD, SkyByte, From-Block-to-Byte, and CMM-H [22, 39, 41, 42] build or evaluate the flash-backed medium. They do not jointly manage graph-ANNS residency, committed-vector materialization, and stall-aware search.

8 Discussion and Limitations

Proxy versus product. The measured path is FPGA `nvme` plus `vmem_sw`, not a shipping CXL.mem SSD. HPS does not fill the 32 GiB BAR from the host CPU; we will

Fig. 9 money plot (hide first):
 stacked `crit_wait` vs. compute at iso-recall;
 twin axis QPS. Lines: Demand, committed-page baseline, full FlashANNS,
 -prefetch, oracle-in-window, PipeANN $T=1$.

Figure 8. Hide at the primary recall floor. The money plot is critical-path NAND wait, not QPS alone. FlashANNS passes only if `crit_wait` approaches the in-window oracle. QPS vs. PipeANN is a named memory-semantic versus block-I/O comparison. Until hide counters pass we write “gap to demand,” not “search cannot see flash.”

Table 5. True-cold T2I-10M, $L=400$, $nq=100$, seed 42, PQ-64 navigation and committed FP rerank. Hide cells (`crit_wait`, score-from-window) stay — until the runtime split. NAND/promote columns stay — until remeasured.

System	Recall@10	QPS	p99 (μs)	NAND reads/q	Promote B/q	Host DRAM
Demand CXL-SSD (P0)	0.944	1.64	1.3×10^6	—	—	1 GiB win.
Page admission	—	—	—	—	—	—
Sync 2-hop	—	—	—	—	—	—
Static wide beam	—	—	—	—	—	—
Miss-chop beam	—	—	—	—	—	—
DiskANN (PQ on)	—	—	—	—	—	—
PipeANN pipe $T=1$	0.925	57.6	1.7×10^4	—	—	PQ nav
Oracle host DRAM ($nq=20$)	0.950	66.5	4.8×10^4	0	—	9.6 GiB pop.
Oracle 1 GiB window ($nq=20$)	0.925	85.8	1.5×10^4	0	—	WS pinned
FlashANNS committed, default layout	—	—	—	—	—	—
FlashANNS committed, co-use layout	—	—	—	—	—	—
FlashANNS + continuous batching	—	—	—	—	—	—

Every — cell is a placeholder. Replace with the measured value under the protocol in §6.1.

Fig. 10 placeholder — two panels:
 (a) when/what: frontier issue vs. commit-time issue and broad vs. committed-page fetch;
 (b) how: default vs. co-use layout, with/without dense coalescing; keep continuous batching as a separate bar.

Figure 9. Wise Prefetcher and continuous-batching ablation. Prefetch variants hold candidate IDs and recall fixed; the utilization panel distinguishes page precision, vector-slot utilization, and extent utilization. **Fill-in:** same recall floor as Figure 8.

not relabel that BAR as the evaluated window. Latency cliffs and QD collapse are still properties of memory-semantic flash.

Hide is the service contract, not a fallback.

A usable ANNS service on this tier means the scoring thread does not wait on NAND: every full-precision distance is taken from bytes already in the window

(`score_from_window` = 100%), and `crit_wait_ns` is DRAM-class. Graph/PQ live in host DRAM; after candidate commitment, wise prefetch materializes the rerank pages and continuous issue schedules those batches across queries. A window miss while scoring is a hide failure we close, not a reason to shrink the claim.

Table 6. Wise Prefetcher decisions at the primary recall floor.

Variant	QPS	Slot use	Extent use
Full: Wise + continuous	—	—	—
Frontier-time issue	—	—	—
Broad frontier payload	—	—	—
Default ID layout	—	—	—
No dense coalescing	—	—	100%
Wise only (—continuous)	—	—	—

Every — cell is a placeholder. Replace with the measured value under the protocol in §6.1.

Fig. 11 — cold reload vs. 1 discarded warmup:
 y : crit_wait, score-from-window, QPS.
 If only warm hides, label it service hide.

Figure 10. True-cold (`vmem_sw cache_used = 0`) versus a discarded warmup then 100 queries. Hide that appears only after warmup is a service property, not a cold-start property. Shared-FAM provisioning stays a discussion model, not this figure.

One-copy is a deployment claim, not a result.

Figure 10b is a provisioning model. Shared FAM is unmeasured. Device IOPS remain shared even with one copy.

DiskANN/PipeANN remain the block-path champions. PQ navigation plus deep aio avoids most full-precision reads. FlashANNS also uses PQ navigation, but materializes a committed FP rerank set through a load/store hierarchy rather than issuing traversal records through block I/O. A $\sim 11\times$ QPS gap to PipeANN $T=1$ at iso-recall is expected and reported.

Hot set and install.all. A 64 MiB shared LFU set raised hit rate and destroyed QPS (window thrash). D3 in the paper is entry pin + soft-pin. True-cold `install.all` did not reproduce a historical $\div 5$ QPS drop; we report traffic, not a fake collapse.

What would change our mind. If HPS makes the FPGA BAR a coherent miss window, we re-measure hide there and keep the same counters. If hide counters never leave NAND-class wait, contribution 3 shrinks to “overlap vs. demand” and the intro must not say the host is unaware of CXL-SSD.

9 Conclusion

Graph ANNS has a well-engineered block-SSD path and a well-engineered CXL-DRAM path. Flash-backed CXL memory is the third deployment point — flash-priced capacity, a memory-semantic window, and a future one-copy hosting story — but it is unusable if treated as slow DRAM or as `mmap'd` DiskANN. FlashANNS is a runtime for that tier: after host-side PQ navigation commits a rerank set, wise prefetch materializes only its missing full-precision vector pages, and continuous batching schedules those precise batches across queries. The opportunity is the medium; the contribution is separating search information from payload movement so exact scoring reads resident bytes while the remaining materialization wait stays explicit.

References

- [1] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K. Aguilera, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. 2020. Can Far Memory Improve Job Throughput?. In *Proceedings of the Fifteenth European Conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA, 1–16. doi:10.1145/3342195.3387522
- [2] Martin Aumüller, Erik Bernhardsson, and Alexander John Faithfull. 2017. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. In *Similarity Search and Applications*. Springer, Cham, Switzerland, 34–49. doi:10.1007/978-3-319-68474-1_3
- [3] Irina Calciu, M. Talha Imran, Ivan Puddu, Sanidhya Kashyap, Hasan Al Maruf, Onur Mutlu, and Aasheesh Kolli. 2021. Rethinking Software Runtimes for Disaggregated Memory. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, New York, NY, USA, 79–92. doi:10.1145/3445814.3446713
- [4] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems*, Vol. 34. Curran Associates, Inc., Red Hook, NY, USA, 5199–5212. <https://proceedings.neurips.cc/paper/2021/hash/2cf49e22106c288c95c9d94bb9b0739a-Abstract.html>
- [5] Rongxin Cheng, Yifan Peng, Xingda Wei, Hongrui Xie, Rong Chen, Sijie Shen, and Haibo Chen. 2024. Characterizing the Dilemma of Performance and Index Size in Billion-Scale Vector Search and Breaking It with Second-Tier Memory. arXiv:2405.03267 doi:10.48550/arXiv.2405.03267
- [6] Compute Express Link Consortium 2023. *Compute Express Link Specification, Revision 3.1*. Compute Express Link Consortium. <https://www.computeexpresslink.org/spec-landing-page-compute-express-link>
- [7] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proceedings of the VLDB Endowment* 12, 5 (2019), 461–474. doi:10.14778/3303753.3303754
- [8] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest

Table 7. Related systems on the dimensions that actually differ. **Fill-in:** keep these columns; add a row only if it changes a column.

System	Full-prec. home	Graph home	Slow path	Capacity assumption	Optimize for	Cache unit
DiskANN / Pi-peANN	NVMe sectors	with vectors	async block I/O	flash is enough	IOPS, PQ trips	4 KB / entry cache
Second-Tier / FaTRQ	CXL/RDMA/NVMe	varies	far load	second tier holds index	4 KB vs. fine read	fine-grained far
CXL-ANNS	all CXL-DRAM	CXL	CXL.mem + DSA	pool holds corpus	~100 ns + near-data	relation cache
Cosmos / PNM	device DRAM	device	on-device cores	multi-device DRAM	offload walk/dist	device place
CXL-SSD HW	NAND HDM	n/a	fault / generic	flash + window	composable SSD, no swap	pages / OS
FlashANNS	CXL-SSD	host DRAM	fault + async fill	flash required; not all-DRAM	residency + stall	committed vector pages

- Neighbor Search. In *IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Los Alamitos, CA, USA, 2946–2953. doi:10.1109/CVPR.2013.379
- [9] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference 2023*. Association for Computing Machinery, New York, NY, USA, 3406–3416. doi:10.1145/3543507.3583552
- [10] Donghyun Gouk, Miryeong Kwon, Hanyeoreum Bae, Sangwon Lee, and Myoungsoo Jung. 2023. Memory Pooling With CXL. *IEEE Micro* 43, 2 (2023), 48–57. doi:10.1109/MM.2023.3237491
- [11] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. 2022. Direct Access, High-Performance Memory Disaggregation with DirectCXL. In *2022 USENIX Annual Technical Conference*. USENIX Association, USA, 287–294. <https://www.usenix.org/conference/atc22/presentation/gouk>
- [12] Juncheng Gu, Youngmoon Lee, Yiwen Zhang, Mosharaf Chowdhury, and Kang G. Shin. 2017. Efficient Memory Disaggregation with Infiniswap. In *14th USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, USA, 649–667. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/gu>
- [13] Hao Guo and Youyou Lu. 2025. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. In *19th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, USA, 171–186. <https://www.usenix.org/conference/osdi25/presentation/guo>
- [14] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning*. PMLR, Online, 3887–3896. <https://proceedings.mlr.press/v119/guo20h.html>
- [15] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *2023 USENIX Annual Technical Conference*. USENIX Association, USA, 585–600. <https://www.usenix.org/conference/atc23/presentation/jang>
- [16] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57
- [17] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. doi:10.1109/TBDATA.2019.2921572
- [18] Myoungsoo Jung. 2022. Hello Bytes, Bye Blocks: PCIe Storage Meets Compute Express Link for Memory Expansion (CXL-SSD). In *14th ACM Workshop on Hot Topics in Storage and File Systems*. Association for Computing Machinery, New York, NY, USA, 45–51. doi:10.1145/3538643.3539745
- [19] Kyungbin Kim, Sungsu Ahn, Wonjung Jeong, Jongmin Kim, Sangun Choi, Minseong Gil, Minseong Kim, Dongha Jung, Yunjay Hong, Haekang Jung, and Yunho Oh. 2026. Bauhaus: Restructuring Vector Database for LLM Retrieval on CXL-Based Tiered Memory. *IEEE Trans. Comput.* 75, 4 (2026), 1309–1322. doi:10.1109/TC.2026.3656215
- [20] Seoyoung Ko, Hyunjeong Shim, Wanju Doh, Sungmin Yun, Jinin So, Yongsuk Kwon, Sang-Soo Park, Si-Dong Roh, Minyong Yoon, Taeksang Song, and Jung Ho Ahn. 2025. Cosmos: A CXL-Based Full In-Memory System for Approximate Nearest Neighbor Search. *IEEE Computer Architecture Letters* 24, 1 (2025), 173–176. doi:10.1109/LCA.2025.3570235
- [21] Ravishankar Krishnaswamy, Magdalen Dobson Manohar, and Harsha Vardhan Simhadri. 2024. The DiskANN Library: Graph-Based Indices for Fast, Fresh and Filtered Vector Search. *IEEE Data Engineering Bulletin* 48, 3 (2024), 20–42. <http://sites.computer.org/debull/A24sept/p20.pdf>
- [22] Miryeong Kwon, Donghyun Gouk, Junhyeok Jang, Jinwoo Baek, Hyunwoo You, Sangyoon Ji, Hongjoo Jung, Junseok Moon, Seungkwan Kang, Seungjun Lee, and Myoungsoo Jung. 2025. From Block to Byte: Transforming PCIe Solid-State Devices With Compute Express Link Memory Protocol and

- Instruction Annotation. *IEEE Micro* 45, 6 (2025), 46–55. doi:10.1109/MM.2025.3581448
- [23] Miryeong Kwon, Sangwon Lee, and Myoungsoo Jung. 2023. Cache in Hand: Expander-Driven CXL Prefetcher for Next Generation CXL-SSD. In *15th ACM Workshop on Hot Topics in Storage and File Systems*. Association for Computing Machinery, New York, NY, USA, 24–30. doi:10.1145/3599691.3603406
- [24] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. 2023. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. Association for Computing Machinery, New York, NY, USA, 574–587. doi:10.1145/3575693.3578835
- [25] Yuri A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473
- [26] Hasan Al Maruf and Mosharaf Chowdhury. 2020. Effectively Prefetching Remote Memory with Leap. In *2020 USENIX Annual Technical Conference*. USENIX Association, USA, 843–857. <https://www.usenix.org/conference/atc20/presentation/al-maruf>
- [27] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit O. Kanaujia, and Prakash Chauhan. 2023. TPP: Transparent Page Placement for CXL-Enabled Tiered-Memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. Association for Computing Machinery, New York, NY, USA, 742–755. doi:10.1145/3582016.3582063
- [28] Jie Ren, Minjia Zhang, and Dong Li. 2020. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 10672–10684. <https://proceedings.neurips.cc/paper/2020/hash/788d986905533aba051261497ecffcb-Abstract.html>
- [29] Zhenyuan Ruan, Malte Schwarzkopf, Marcos K. Aguilera, and Adam Belay. 2020. AIFM: High-Performance, Application-Integrated Far Memory. In *14th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, USA, 315–332. <https://www.usenix.org/conference/osdi20/presentation/ruan>
- [30] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. 2022. Results of the NeurIPS’21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *NeurIPS 2021 Competitions and Demonstrations Track*. PMLR, Online, 177–189. <https://proceedings.mlr.press/v176/simhadri22a.html>
- [31] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. arXiv:2105.09613 doi:10.48550/arXiv.2105.09613
- [32] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc., Red Hook, NY, USA, 13748–13758. <https://proceedings.neurips.cc/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html>
- [33] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. 2023. Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices. In *56th IEEE/ACM International Symposium on Microarchitecture*. Association for Computing Machinery, New York, NY, USA, 105–121. doi:10.1145/3613424.3614256
- [34] Chenxi Wang, Yifan Qiao, Haoran Ma, Shi Liu, Yiyang Zhang, Wenguang Chen, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. 2023. Canvas: Isolated and Adaptive Swapping for Multi-Applications on Remote Memory. In *20th USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, USA, 161–179. <https://www.usenix.org/conference/nsdi23/presentation/wang-chenxi>
- [35] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 2614–2627. doi:10.1145/3448016.3457550
- [36] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–27. doi:10.1145/3639269
- [37] Lingfeng Xiang, Zhen Lin, Weishu Deng, Hui Lu, Jia Rao, Yifan Yuan, and Ren Wang. 2024. Nomad: Non-Exclusive Memory Tiering via Transactional Page Migration. In *18th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, USA, 19–35. <https://www.usenix.org/conference/osdi24/presentation/xiang>
- [38] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 545–561. doi:10.1145/3600006.3613166
- [39] Shao-Peng Yang, Minjae Kim, Sanghyun Nam, Juhung Park, Jin yong Choi, Eeye Hyun Nam, Eunji Lee, Sungjin Lee, and Bryan S. Kim. 2023. Overcoming the Memory Wall with CXL-Enabled SSDs. In *2023 USENIX Annual Technical Conference*. USENIX Association, USA, 601–617. <https://www.usenix.org/conference/atc23/presentation/yang-shao-peng>
- [40] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, USA, 521–538. <https://www.usenix.org/conference/osdi22/presentation/yu>
- [41] Jianping Zeng, Shuyi Pei, Da Zhang, Yuchen Zhou, Amir Beygi, Xuebin Yao, Ramdas Kachare, Tong Zhang, Zongwang

- Li, Marie Nguyen, Rekha Pitchumani, Yang Soek Ki, and Changhee Jung. 2025. Performance Characterizations and Usage Guidelines of Samsung CMM-H. *IEEE Micro* 45, 5 (2025), 94–102. doi:10.1109/MM.2025.3591577
- [42] Haoyang Zhang, Yuqi Xue, Yirui Eric Zhou, Shaobo Li, and Jian Huang. 2025. SkyByte: Architecting an Efficient Memory-Semantic CXL-based SSD with OS and Hardware Co-design. In *2025 IEEE International Symposium on High Performance Computer Architecture*. IEEE, Los Alamitos, CA, USA, 577–593. doi:10.1109/HPCA61900.2025.00051
- [43] Tianqi Zhang, Flavio Ponzina, and Tajana Rosing. 2026. FaTRQ: Tiered Residual Quantization for LLM Vector Search in Far-Memory-Aware ANNS Systems. arXiv:2601.09985 doi:10.48550/arXiv.2601.09985
- [44] Yuhong Zhong, Daniel S. Berger, Carl A. Waldspurger, Ryan Wee, Ishwar Agarwal, Rajat Agarwal, Frank Hady, Karthik Kumar, Mark D. Hill, Mosharaf Chowdhury, and Asaf Cidon. 2024. Managing Memory Tiers with CXL in Virtualized Environments. In *18th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, USA, 37–56. <https://www.usenix.org/conference/osdi24/presentation/zhong-yuhong>
- [45] Zhe Zhou, Yiqi Chen, Tao Zhang, Yang Wang, Ran Shu, Shuotao Xu, Peng Cheng, Lei Qu, Yongqiang Xiong, Jie Zhang, and Guangyu Sun. 2024. NeoMem: Hardware/Software Co-Design for CXL-Native Memory Tiering. In *57th IEEE/ACM International Symposium on Microarchitecture*. IEEE, Los Alamitos, CA, USA, 1518–1531. doi:10.1109/MICRO61859.2024.00109